

# Interview Revision Notes

[Python](#) | [SQL](#) | [Git & GitHub](#)

---

1. Python — `filter()`, `map()`, `sorted()`
2. SQL — Correlated Subqueries
3. Git & GitHub — Core Workflow

# Python — filter(), map(), sorted()

## 1. filter()

Used to keep/remove elements based on a condition.

```
employees = list(filter(  
    lambda employee: employee["salary"] > 70000,  
    employees  
))
```

**Remember:** filter() → select

## 2. map()

Used to transform/extract values.

```
employees_names = list(map(  
    lambda employee: employee["name"],  
    employees  
))
```

**Remember:** map() → transform

## Common Pattern

```
filter → select required objects  
map    → extract/transform something
```

## Example

```
employees = list(filter(
    lambda employee:
        employee["department"] == "IT"
        and employee["salary"] > 70000,
    employees
))

employees = list(map(
    lambda employee: employee["name"],
    employees
))
```

Output

```
['Alice', 'Charlie']
```

### 3. map() with Full Dictionary

To preserve the complete dictionary while changing one value:

```
employees = list(map(
    lambda employee: {
        **employee,
        "salary": employee["salary"] * 1.10
    },
    employees
))
```

Important: `**employee` copies the existing dictionary fields, then `"salary": employee["salary"] * 1.10` overwrites the salary.

### 4. sorted()

Sort employees based on salary:

```
employees = sorted(
    employees,
    key=lambda employee: employee["salary"],
    reverse=True
)
```

- `key=` → what to sort by
- `reverse=True` → descending
- Without `reverse=True` → ascending

#### Interview Pattern

```
sorted(data, key=lambda x: value, reverse=True)
```

# SQL — Correlated Subqueries

We used Employee\_1 with columns:

```
id | name | salary | departmentId
```

## 1. Salary Greater Than Department Average

```
SELECT e.name
FROM Employee_1 e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee_1 e2
    WHERE e2.departmentId = e.departmentId
);
```

### Key Concept: Correlated Subquery

The inner query depends on the current row of the outer query:

```
WHERE e2.departmentId = e.departmentId
```

**Meaning:** "Calculate the average salary for the same department as the current employee."

### Why e and e2?

They are aliases representing two different references to the same table.

```
Employee_1 e
Employee_1 e2
```

- e → current employee from outer query
- e2 → employees being examined by the inner query

## 2. Highest Salary in Each Department

```
SELECT e.name
FROM Employee_1 e
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM Employee_1 e2
    WHERE e2.departmentId = e.departmentId
);
```

### Important Pattern

For highest salary:

```
salary = MAX(salary)
```

Not:

```
salary > MAX(salary)
```

Because nothing can be greater than the maximum.

# Git & GitHub — Core Workflow

## Basic Workflow

```
git status
git add .
git commit -m "change"
git push
```

## Flow

```
Working Directory → git add → Staging Area → git commit → Local
Repository → git push → GitHub
```

## git status

Shows the current state of your files. For example:

```
modified: app.py
modified: README.md
```

It helps you see which files have changes and their Git state.

## What Does “Staged” Mean?

**Staged** = selected and ready to be included in the next commit.

```
git add app.py
```

Now `app.py` is staged.

## git commit

```
git commit -m "add login feature"
```

Saves the staged changes into your local repository.

## git push

```
git push
```

Uploads your local commits to the remote repository/GitHub.

### Interview Answer

Commit saves changes to the local repository, while push uploads those commits to the remote repository such as GitHub.

## Unstage a File

If you accidentally do:

```
git add app.py
```

and want to remove it from staging:

```
git restore --staged app.py
```

This does not delete your changes. It simply moves the file from:

```
Staging Area  
↓  
Working Directory
```